

Temporary Destruction (CTF@CIT 2026)

Description:

I hear something....

Overview

Webapp reflects what ever user input. Due to misuse of `render_template_string` of dev, the webapp is vulnerable to `Server-side template injection`. Abuse the vuln to read flag at `/tmp/flag.txt`

Source code and Vulnerabilty analysis

The server gets data from user and validate it with this condition: `BLOCKED = re.compile(r'__\w+__')`, the dev tends to block keyword like `dunder method`: `__globals__`, `__class__`,...

However, blacklist is never enough =))

Instead of inserting the user's text directly into the html content, backend uses `render_template_string` which is infamous for sink of SSTI.

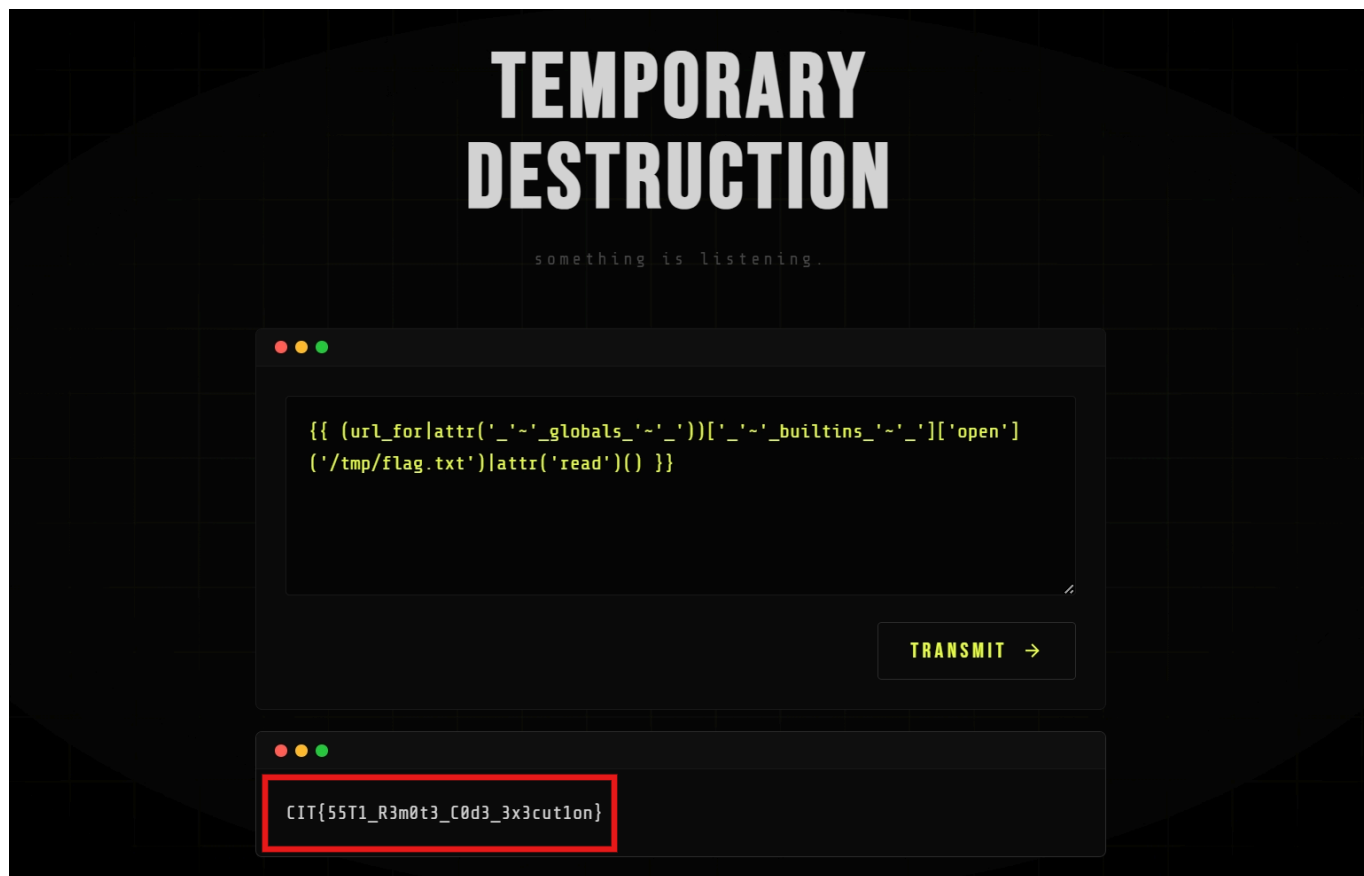
```
temporary-destruction > app.py > index
72
73
74 app.route('/', methods=['GET', 'POST'])
75 if index():
76     raw_input = ''
77     output = None
78     is_error = False
79
80     if request.method == 'POST':
81         raw_input = request.form.get('user_input', '')
82
83         if BLOCKED.search(raw_input):
84             output = 'rejected.'
85             is_error = True
86         else:
87             try:
88                 output = render_template_string(raw_input)
89             except Exception:
90                 output = 'error.'
91                 is_error = True
92
93     return render_template_string(
94         PAGE,
95         raw_input=raw_input,
96         output=output,
97         is_error=is_error
98     )
99
100
101 if __name__ == '__main__':
    ...

temporary-destruction > app.py > ...
40 <textarea
41     name="user_input"
42     id="inp"
43     rows="5"
44     spellcheck="false"
45     >{{ raw_input }}</textarea>
46 </div>
47 <div class="actions">
48     <button type="submit">
49         <span>TRANSMIT</span>
50         <svg width="16" height="16" viewBox="0
51     </button>
52 </div>
53 </form>
54 </div>
55 </div>
56
57 {% if output is not none %}
58 <div class="response {% if is_error %}bad{% endif %}">
59     <div class="card-bar">
60         <div class="bar-dots"><i></i><i></i><i></i><i></i></div>
61     </div>
62     <div class="response-inner">
63         <pre>{{ output }}</pre>
64     </div>
65 </div>
66 {% endif %}
67 </div>
68
69 <script src="/static/js/main.js"></script>
```

The code renders/runs the malicious code user submits, gets the result and puts back into the html content.

Exploitation

I use `~` to concatenate `__globals__` into `__globals__`. Or you can also use hex encode as well



Flag: ***CIT{55T1_R3m0t3_C0d3_3x3cut1on}***